INFO-H415 : Graph databases

BERTHION Antoine - 566199
EZERSKIS Andrius - 542698
SPEILERS Capucine - 540555
TRUONG Nha - 576343

1. Introduction

For this project, the advanced database topic chosen is **Graph Databases** with two different tools: **Neo4j** and **ArangoDB**. Oriented graph databases are an alternative approach to classical models such as relational databases. They are based on graph theory, which makes them particularly suitable for representing sets of strongly interconnected entities.

We decided to choose **Neo4j** because it is widely used in the domain of graph databases and is open source. We compared it with **ArangoDB** because both share some properties but differ in how they implement the graph structure. While **Neo4j** is a native graph database, **ArangoDB** is a multi-model database. Comparing the two tools is interesting to see which one is more suitable for our graph database on individuals infected with covid.

In this document, we will cover the database and dataset choice, the methods, and the results for the comparison of **Neo4j** and **ArangoDB**.

1.1. Dataset generation

In order to benchmark the two technologies, we have generated different datasets, containing names, countries of origin and whether each person has covid or not. As for the size of those datasets, it varies from 10 000 to 1 million nodes, and from 30 000 edges to 3 million edges. The goal of this increasing size is to see how the two technologies handle data scaling, as well as how it handles large collections of data.

2. Technical background

This section introduces some technical and mathematical notation from graph theory, providing the fundamentals for understanding graph databases.

2.1. Nodes and relationships

An oriented graph is defined as a pair:

$$G = (V, E) \tag{1}$$

where V is a finite set of vertices, and $E \subseteq V \times V$ is a set of oriented edges.

In graph databases, we talk about **nodes** instead of vertices and **relationships** instead of edges. A node represents an *entity* (discrete object) of a domain, while a relationship describes how a *source node* and a *target node* are connected.

2.2. Properties

In order to store additional information, two attribute functions are introduced:

$$\phi : V \to \mathcal{A}_V \tag{2}$$

$$\phi : E \to \mathcal{A}_E \tag{3}$$

where $\mathcal{A}_V$ and $\mathcal{A}_E$ denote respectively the sets of possible attributes for vertices and edges.

Intuitively, we can link this notation to the notion of multiple-entry tables in classical databases. Such a model is called a property graph. Properties are key-value pairs used to store data on both nodes and relationships.

2.3. Traversals and graph-specific queries

Querying a graph database is different from querying a relational database: instead of joining tables, queries rely on traversals that explore the graph in order to find answers. A query corresponds to traversing a graph by visiting nodes V and following relationships E according to specific rules. Typically, only a subset of the graph is visited.

To describe how traversals work, we introduce three notions: neighborhood, paths, and k-hop neighbors.

2.3.1. Neighborhood

The minimal step in traversals is going from one node to its immediate neighbors. The out-neighborhood of a vertex $v \in V$ is defined by:

$$\Gamma(v) = \{u \in V \mid (v, u) \in E\} \tag{4}$$

which is the set of all vertices directly reachable from v via a single outgoing edge.

2.3.2. Paths

More complex traversals consist of following a sequence of edges. A path of length k is a sequence

$$(v_0, v_1, ..., v_k) \tag{5}$$

such that $(v_i, v_{i+1}) \in E$ for all $0 \leq i < k$.

2.3.3. k-hop neighbors

A vertex u is a **k-hop neighbor** of v if there exists a path of length k from v to u. The set of all k-hop neighbors of v is called its **k-hop neighborhood**. Note that 1-hop neighbors correspond exactly to $\Gamma(v)$, the direct neighbors of v. This concept is fundamental for queries that explore relationships at varying distances, such as finding friends-of-friends in a social network (2-hop) or analyzing influence propagation over multiple steps.

3. Database choice

3.1. Graph vs. relational databases

Graph databases and relational databases are designed for different use cases. Understanding the strengths and limitations of each technology helps explain why their relative efficiency varies depending on the use case.

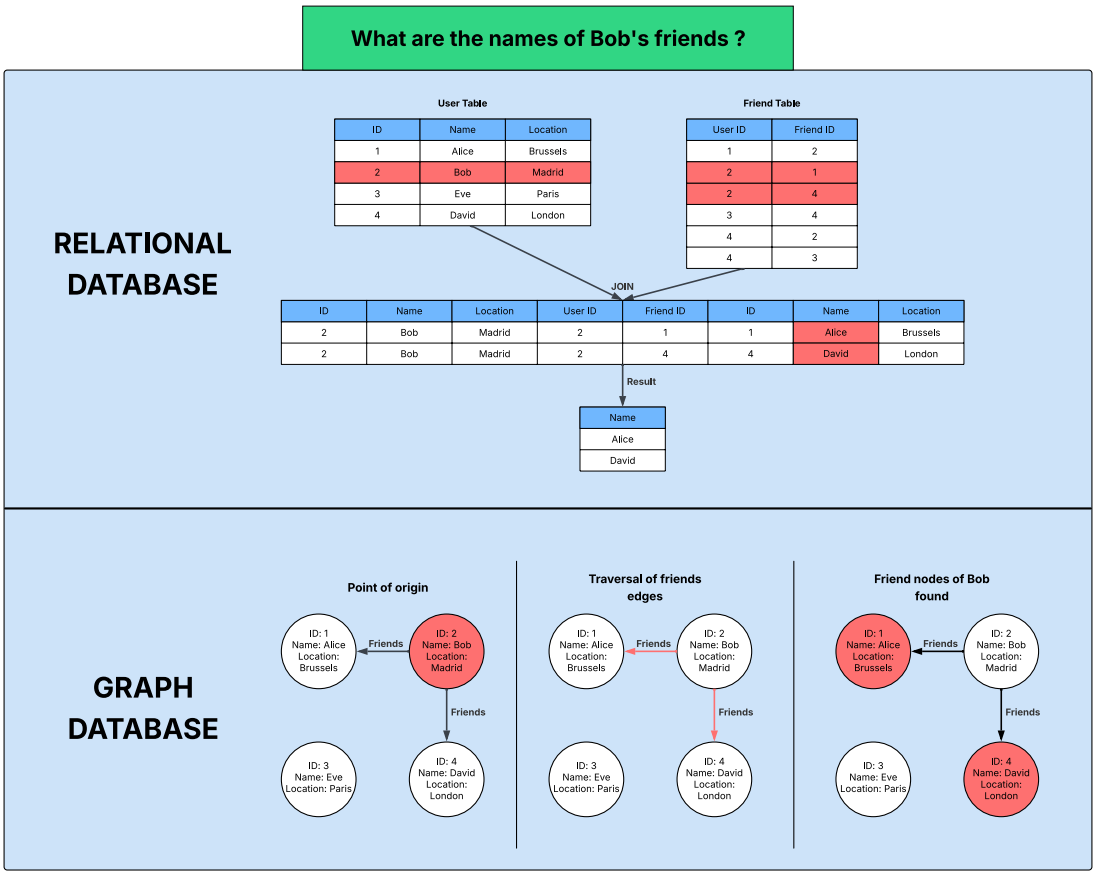


Figure 1: Comparison of relational and graph databases

3.1.1. Relational databases

The model behind relational databases uses data tables organizing information in rows and columns, where columns contain specific attributes of the data, and rows are individual pieces of the data.

To understand what goes on when querying a relational database, let's take a look at the example shown in Figure 1.

Suppose we have a user table containing the ID, name and location of the users of some service. Additionally, there is a friends table containing pairs of user IDs representing friendships.

Now, let's say we want to query something like *What are the names of Bob's friends?* The database management system would:

1. Find Bob's row in the users table to retrieve his ID
2. Find every row in the friends table where Bob's ID appears
3. Join each friend ID from those rows back to the users table

Then, the database management system will return the names of Bob's friends.

Solving that simple query requires multiple table scans and joins, leading to a large and expensive data structure. From a graph perspective, this query is equivalent to finding the 1-hop neighborhood of

Bob. Note that the number of joins grows linearly with k, the number of hops: a 2-hop query requires joining three tables, a 3-hop query requires four, and so on. Moreover, as the depth of our queries increases, the memory consumed for intermediate results will grow rapidly and impact performance very early.

3.1.1.1. Strengths

First of all, relational database systems organize data into tables, each one consisting of columns (fields) and rows (records). This structure allows for a consistent format and makes it easy to store and retrieve data.

Moreover, tools like primary keys, foreign keys and unique constraints allow to keep the data consistent and accurate, preventing the entry of invalid or redundant data.

SQL, the language used to interact with those systems, is a powerful tool for querying and managing data. It allows to perform complex queries, join multiple tables, filter data, and so on.

3.1.1.2. Limitations

Although relational database systems can handle a fair amount of data, they face significant performance bottlenecks as datasets grows larger and queries become more complex.

The fixed nature of relational database systems can be a bottleneck since adding new columns or changing relationships between tables can be complex and may require significant changes to applications and/or downtime. Additionally, complex joins across multiple tables also often leads to bottlenecks as the time taken to run queries increases significantly along the size of the datasets, as mentioned before.

3.1.2. Graph databases

Graph databases, on the other hand, use graph-based structures with attributes, relationships and objects to represent data. They allow for a better representation of connected data.

Back to our example, each user would be represented as a node with their ID, name, and location as properties of that node. Then, to represent the friendship between users, we simply introduce the relationship *friends with*. With this structure, retrieving the friends of a user requires looking at all the edges labeled *friends with* that are connected to that user. Then the nodes on the other end of those edges are retrieved, and the names of those neighbors can be returned.

Unlike relational database, graph databases are optimized for traversal operations. Finding k-hop neighbors simply requires following k edges, without the overhead of joins or temporary data structures that relational databases require. In other words, the natural graph representation makes graph traversals significantly more efficient by eliminating the need for complex join operations.

3.1.2.1. Strengths

Graph databases are very efficient when handling highly connected data, as the structure of graphs allows to represent relationships, and in turn allows to handle more effectively the scaling of data.

Additionally, traversing the graph to perform queries is usually much faster than in a relational database, as nodes are often highly connected. Tasks like recommendation systems, fraud detection or social network analysis hugely benefit from this feature, as it often boils down to understanding connections between entities, meaning we have to traverse relationships in graphs.

Moreover, the flexible structure of graphs allows to adapt and change the structure and schema of graph models, without having to alter current functionality. This is useful in a business environment that is constantly evolving, and increases maintenance and productivity tremendously.

3.1.2.2. Limitations

Naturally, as for any technology or tool, there are limitations to graph databases. For example, some scenarios and use cases simply do not fit well with the inner workings of graph databases, such as storing entity properties with extremely large values, like Binary Large Objects (BLOBs, often multimedia objects), or Character Large Objects (CLOBs, collections of character data).

In addition, graph databases are primarily designed for single-server architectures, which makes it harder to scale as the data grows larger.

3.2. Neo4j

Neo4j is a *native graph database management system*, meaning that it implements a true graph model all the way down to the storage level. The data is stored as nodes, relationships connecting them and properties associated with both nodes and relationships.

It is currently the most widespread graph-oriented database management system. Neo4j directly implements the *property graph* model described earlier: each node and relationship has a set of atrributes (ϕ_V, ϕ_E).

Queries are expressed using **Cypher**, Neo4j declarative query language similar to SQL, but specially designed and optimized for graphs. This language allows to express patterns of the form:

$$(v_1) - [e] \rightarrow (v_2) \tag{6}$$

which correspond exactly to the pair $(v_1, v_2) \in E$.

Neo4j provides advanced features including the ability to deliver fast responses even for highly complex queries, often faster than relational databases. It is designed to scale efficiently to support massive growth in data and users, while optimizing costs. Neo4j also relies on a flexible property graph model and ensures full ACID compliance, that guarantees transactional integrity for mission-critical workloads.

3.2.1. Strengths

Neo4j is currently the most widely recognized and adopted graph database system, ranking significantly higher in popularity than ArangoDB according to DBMS popularity indices[1]. Because Neo4j is more broadly used, it benefits from a richer ecosystem of learning resources and community support compared to ArangoDB.

Neo4j is a native graph database. This means that its storage engine, data model, and query mechanisms are built specifically for graph structures. The single-model design allows specialization in graph operations and optimizations.

This specialization is also reflected in its query language, Cypher, a declarative language designed specifically for expressing graph patterns in a clear and readable way. Cypher is intuitive and expressive, especially for complex or highly connected queries.

Additionally, Neo4j follows the property graph model, storing data as key-value pairs on both nodes and relationships. With Neo4j Browser, it provides a robust environment for visualizing and interacting with graph data, offering immediate graphical insight into data structures and query results.

3.2.2. Limitations

Even though Neo4j can scale reads, by sharing the read load across slaves, only the master instance in the cluster can handle the writes. Slaves can receive write requests, but will forward them to the master node anyway, so writing to the master instance is faster than writing to a slave instance.

[1] DB-Engines benchmarks methods [https://db-engines.com/en/ranking_definition]

Additionally, Neo4j has sharding support, but some queries and operations perform very poorly on sharded Neo4j databases, and only work with some specific versions of the Cypher query language.

3.3. ArangoDB

ArangoDB is a multi-model database system that supports JSON documents, key-value pairs and graphs data models within a single, unified engine.

In its graph model, ArangoDB also follows as Neo4j the classical mathematical representation of a graph as a pair (V, E). Vertices are stored as JSON documents in document collections, while edges are sorted in specialized edge collections that explicitly define the relationships between vertices through `_from` and `_to` attributes. This structure supports the explicit modeling of directed and undirected graphs, as well as labeled and attributed edges.

Queries are expressed in AQL (*Arango Query Language*), mainly a declarative language designed to work uniformly across all supported data models. It allows the combination of document retrieval, key-value access, and graph traversal operations within a single query. In particular, AQL provides native constructs for graph navigation, allowing multi-hop traversals to be expressed directly in the query language.

A graph traversal in AQL can be expressed as:

```
FOR v, e, p IN 1..k OUTBOUND s GRAPH G RETURN v.
```

This syntax corresponds directly to the neighborhood

$$\Gamma^k(s) \tag{7}$$

defined mathematically in the Equation 4.

3.3.1. Strengths

The main strength of ArangoDB is that it is a multi-model system, meaning it can handle multiple data models, such as documents, key-value pairs, as well as graphs. This allows for a strong adaptability as the data requirements for a project can change and evolve.

Its query language, AQL, allows for unified queries to be able to handle the different types of data. Moreover, for our use case (graph databases), ArangoDB excels at managing interconnected data, where there are multiple complex relationship mappings.

Finally, managing all data models with a single engine optimizes resource utilization, leads to reduced costs for infrastructure and simplifies database management.

3.3.2. Limitations

Despite all the advantages of using ArangoDB, the technology is newer than Neo4j, and the community is smaller than Neo4j's one, resulting in less features added overtime, and is thus less used than the more popular graph database systems.

Moreover, some integrations, like GraphQL, are limited, and the schema (used to describe what data can be queried from the GraphQL API) needs to be generated manually, instead of using the graph database's schema directly.

4. Methods

4.1. Dataset

As mentioned above, we chose to generate our own datasets. This choice was motivated by our limits in terms of device: most online datasets we found were either too large for our experimental setup or required extensive modification and cleaning before use. Moreover, simply reducing the size of large graph datasets is not straightforward, as both nodes and edges must be carefully selected to maintain meaningful graph structure and connectivity.

The dataset structure we created is based on a social graph of people during Covid:
- Nodes: `id, firstName, lastName, age, hasCovid, origin`
- Edges: `id, from, to, exposure`

This structure allows us to filter nodes by properties and perform graph traversal queries.

4.1.1. Graph sizes

To assess scalability and performance evolution, we evaluated each graph database using three different graph sizes:
- 10K nodes with 30K edges
- 30K nodes with 150K edges
- 1M nodes with 3M edges

The smallest dataset allows us to measure performance on a relatively simple graph, the intermediate dataset introduces higher density and more complex graph traversals and the larger dataset stresses the systems in terms of memory usage and scalability.

The edge-node ratio is kept relatively consistent to preserve comparable graph density. This ensures that performance differences are due to scale changes rather than structural ones.

4.2. Benchmark queries

There are four main actions that can be performed on databases : reading, writing, updating and removing. We designed 6 queries covering those use cases alongside with graph-specific operations, as discussed before.

4.2.1. Data insertion

Data insertion is fundamental for databases as it reflects how efficiently the system handles data ingestion and, in our case, graph construction. Measuring insertion time allows us to assess write performance and the impact of graph size on bulk data loading. This is particularly relevant for dynamic use cases with frequent graph updates.

4.2.2. Graph traversals

Queries 1 and 2 covers 1-hop and 2-hops neighborhood sensing. These queries represents fundamental graph operations and allow us to evaluate how efficiently each database handles multi-level relationship exploration.

Query 1 Count people who have at least one contact with Covid
Query 2 Count people with no Covid cases in their 2-degree circle

4.2.3. Edge filtering

Query 3 is similar to Query 1 but focuses specifically on edge filtering. By restricting the query to relationships with particular properties, we can evaluate how efficiently each database handles relationship filtering.

Query 3 Count healthy people at risk from CLOSE contacts

4.2.4. Node filtering

Query 4 helps us evaluate a simple node filtering query.

Query 4 Count older people (age > 65) infected with Covid

4.2.5. Graph updating

Query 5 updates nodes properties, allowing us to evaluate the writing capabilities of each tool.

Query 5 Covid propagation in 1-hop neighborhoods

4.2.6. Node removal

Query 6 tests the efficiency of tools with item removal.

Query 6 Node deletion for Covid fatalities

4.3. Experimental setup

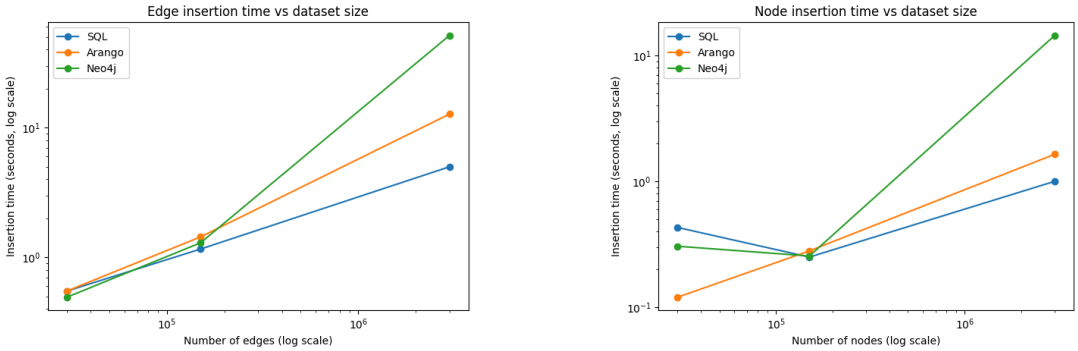
Experiments were conducted on a machine equipped with a i7 (12k) CPU with 64 GB of RAM.

Each query was first executed one time as a warm-up in order to populate the cache. Then the query was executed 5 times and the results were used for the analysis. Queries that did not complete within a reasonable time frame were executed fewer times.

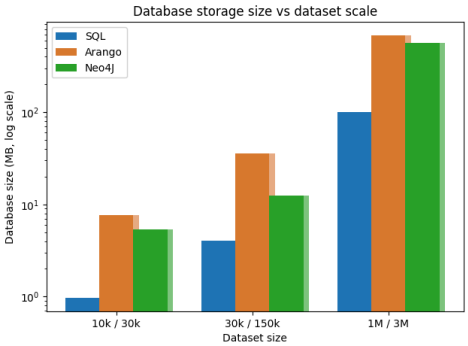
Insertion performance was evaluated separately. For each dataset size, the data insertion process was executed five times, allowing us to ensure more reliable average measurements.

5. Results and discussion

5.1. Data insertion



Results for edge insertion on the smallest and intermediate datasets show similar performance. However, for the largest dataset, Neo4j and ArangoDB are respectively around 10 times and 2 times slower than SQL. This behavior can be explained by differences in data models. In SQL, edge insertion corresponds to inserting rows into a relational table. In contrast, ArangoDB stores edges as documents in a collection, while Neo4j maintains its own graph structure, which implies higher insertion costs, especially at larger scales.



SQL uses less storage space, especially at smaller scales, while ArangoDB uses the most space. Again, this can be explained by differences in data models. SQL relies on simple relational tables, ArangoDB stores data as collections of JSON documents along with indexes, and Neo4j implements its own graph structure. Remark that, for the intermediate dataset, ArangoDB uses more space than expected compared to Neo4j. This suggests that the number of edges has a greater impact on storage usage in ArangoDB than in Neo4j.

5.2. Queries results

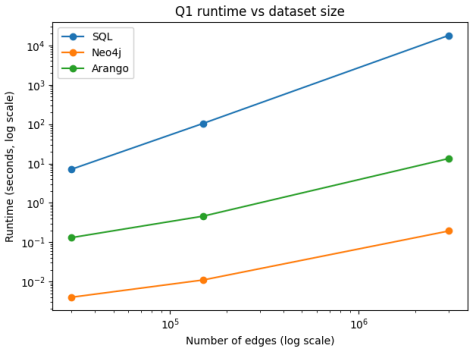


Figure 5: Graph traversal: 1-hop

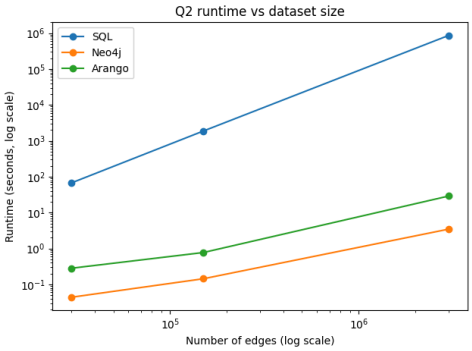


Figure 6: Graph traversal: 2-hop

For the direct neighborhood (1-hop) query, SQL is nearly 100 to 1,000 times slower than ArangoDB, while ArangoDB is slightly more than 100 times slower than Neo4j.

For the 2-hop neighborhood query, SQL is again significantly slower, ranging from nearly 100 to 10,000 times slower than ArangoDB. ArangoDB performs slightly worse than Neo4j on small datasets (less than 10 times slower), but is about 100 times slower on the largest dataset.

SQL results can be explained by the need of expensive joins to solve those queries. Remark that the runtime with SQL grows slightly slower than linearly with the number of edges.

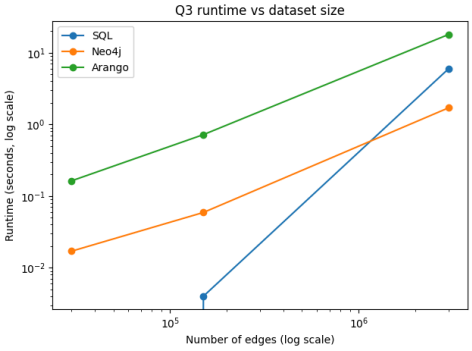


Figure 7: Edge filtering

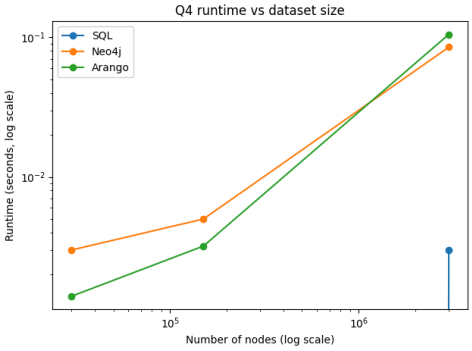


Figure 8: Node filtering

Figure 7 shows that SQL query is faster on average than ArangoDB and Neo4j. This is because it consists of a simple join with some property filters, something relational databases are highly optimized for.

SQL engines can use indexes to efficiently identify relevant rows and handle everything in a bulk-oriented manner. Graph databases, on the other hand, are designed for exploring complex relationships and multi-hops traversals. Even something as simple as checking direct neighbors in Neo4j or ArangoDB involves pattern matching and going through edges one by one, which adds overhead. Although we observe that Neo4j and ArangoDB follows a similar runtime evolution, Neo4j outperfoms ArangoDB by a constant 10 times factor for filtering neighbors.

For straightforward, one step queries like this, SQL naturally outperforms graph databases, while graph databases really shine when deeper or more complex graph traversals are required.

Figure 8 shows that SQL performs particularly well on node filtering, as this operation reduces to a simple select query on the nodes table. In contrast, Neo4j is approximately 2 times slower than ArangoDB at small scale and about 2 times faster on the largest dataset. The additional overhead introduced by the graph-specific data structures of Neo4j could explain that small datasets run slower.

These results indicate that when the use case does not primarily rely on relationships or graph traversals, a relational database such as SQL can be the most efficient option.

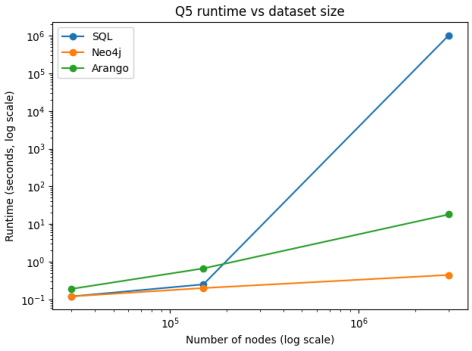


Figure 9: Node updating

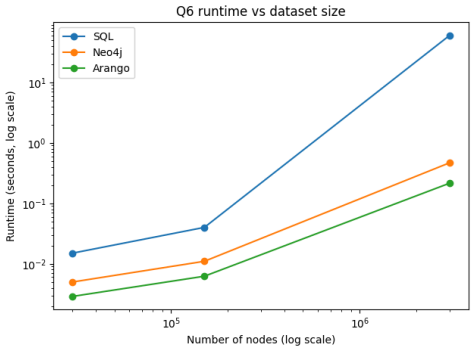


Figure 10: Node deletion

Results for node updates show that Neo4j and ArangoDB stay linear with the size of the graph. ArangoDB, Neo4j and SQL all starts with similar runtime, however ArangoDB ends up with a runtime 100 times slower than Neo4j, while SQL degrades significantly on the largest dataset, where it becomes approximately 10,000,000 times slower than Neo4j.

This behavior can be explained by the nature of Query 5, which models a propagation process over 1-hop neighborhoods. Graph databases such as Neo4j and ArangoDB are specifically optimized for traversing relationships and updating connected nodes, resulting in proportional scalability.

In contrast, implementing such propagation in SQL requires costly join and update operations over large tables, which do not scale well at larger graph sizes. As the dataset grows, the increasing number of joins, updates, and index maintenance operations leads to a dramatic performance degradation.

Moreover, Figure 10 shows that, for node removals, ArangoDB outperforms Neo4j by a factor of 2 to 5 on small datasets and by a factor 2 again on the largest dataset. This comparison could be attributed to differences in internal graph representations and the way each database maintains relationships and indices during deletions.

Regarding SQL, its execution time remains approximately 10 times slower than ArangoDB at small scales but degrades dramatically on the largest dataset, where it becomes nearly 1,000 times slower. This behavior can be explained by the need to perform costly join operations and bulk updates over large relational tables when deleting nodes and maintaining referential integrity, which scale poorly for graph-like deletion operations.

6. Conclusion

Benchmarking Neo4j and ArangoDB gave us a pretty clear overview of each database's specs, while comparing them alongside a relational database showed us both the strengths and limits of the graph model.

Neo4j is definitely the winner for graph-specific operations. It outperformed both ArangoDB and SQL when it came to graph traversals, especially for 1-hop and 2-hop queries. This makes sense since Neo4j is a native single-model graph database built on graph structures from the ground up. It also handled node updates really well, maintaining linear performance even when we scaled up to a million nodes.

ArangoDB didn't perform as well as Neo4j for pure graph traversals, but it still did a decent job compared to SQL in that graph context. The main advantage here is that ArangoDB is multi-model, offering flexibility to handle different types of data models. This could be particularly useful for certain use cases, as shown by the node filtering query results. We could imagine scenarios where lots of node filtering operations are performed while relationships are still important enough to justify avoiding a pure relational database.

Interestingly, SQL wasn't completely out of the race. For simple operations like filtering nodes by properties or basic edge filtering, SQL actually beat both graph databases. It's only when queries get more complex and graph-specific (like multi-hop traversals) that SQL starts to struggle badly.

So what should you actually use? On one hand, Neo4j should be preferred if the application relies heavily on relationships and graph traversals. This will typically be the case for social network apps, fraud detection or recommendation systems - basically the typical graph use cases. On the other hand, ArangoDB should be picked if the application could benefit from that multi-model feature and if its workload involves mixed operations across different data models. Though realistically, ArangoDB would rarely be chosen over Neo4j for pure graph workloads. And finally, graph databases should be avoided if the application has nothing to do with graphs, as shown by SQL outperforming both for queries not tightly related to graph operations.

Bottom line: there's no universal solution. Pick the right tool for the job based on what your application actually needs.